Backend Matchmaking System Documentation

📋 Overview

This backend provides a complete matchmaking and discovery system for your social profile management and mutual match-making application. It includes sophisticated matching algorithms, user interactions, and discovery features.

📐 Architecture

Models

User Model (`models/User.js`)

Enhanced user schema with comprehensive matchmaking fields:

Basic Profile:

- email, password, username
- bio, about
- profilePicture, coverImage, photos

Personal Details:

- pronouns, gender, interestedIn
- dateOfBirth, age (auto-calculated)
- workTitle, workCompany
- educationSchool, educationDegree
- height

Location:

- Geo-coordinates (2dsphere index for proximity search)
- city, state, country, displayLocation

Lifestyle:

- smoking, drinking, exercise, diet

Interests:

- Array of interest tags (max 20)

Relationship Goals:

- Casual dating, Long-term, Marriage, Friendship, Not sure yet

Match Preferences:

- ageRange: {min, max}
- distanceRange (km)
- genderPreference
- dealBreakers, mustHaves

Discovery Settings:

- isActive (show in discovery)
- Visibility settings for age, distance, lastActive

Activity Tracking:

- lastActive, isOnline
- profileCompleteness (0-100, auto-calculated)

Visibility Controls: All fields have individual visibility settings (public/private)

Match Model (`models/Match.js`)

Tracks all user interactions:

Interaction Types:

- `liked` - User swiped right
- `passed` - User swiped left
- `super_liked` - User super liked
- `matched` - Mutual match
- `blocked` - User blocked

Fields:

- fromUser (who performed action)
- toUser (who received action)
- status
- isMutual (boolean)
- matchScore (compatibility 0-100)
- conversationStarted
- lastMessageAt

Indexes:

- Compound index on fromUser + toUser (prevents duplicates)
- Indexes for efficient match queries

Static Methods:

- `checkMutualMatch(userId1, userId2)` - Check if users matched
- `createMutualMatch(userId1, userId2, score)` - Create mutual match
- `getUserMatches(userId, limit)` - Get user's matches

- `getUsersWhoLikedMe(userId)` - Get incoming likes
- `hasInteracted(fromUserId, toUserId)` - Check interaction exists

🧮 Matching Algorithm

Compatibility Score Calculation

The algorithm calculates a **0-100 compatibility score** based on weighted factors:

Weight Distribution:

```javascript
{
  age: 15%,
  location: 20%,
  interests: 25%,
  lifestyle: 20%,
  relationshipGoals: 20%
}
```

1. Age Compatibility (15%)

- Checks if target age is within user's preferred range
- Scores higher for ages closer to range midpoint
- Returns 0 if outside range

2. Location Compatibility (20%)

- Uses Haversine formula for distance calculation
- Linear decrease with distance
- Returns 0 if beyond maxDistance preference

3. Interests Compatibility (25%) - Highest Weight

- Calculates Jaccard similarity (common / union)
- Bonus multiplier for many common interests
- Returns 30 (not 0) if no interests listed

4. Lifestyle Compatibility (20%)

- Compares: smoking, drinking, exercise, diet
- Exact match: 100 points
- Compatible: 70 points (e.g., "Never" + "Socially" smoking)
- Somewhat compatible: 40 points
- Not compatible: 10 points

5. Relationship Goals (20%)

- Exact match: 100
- Compatible goals: 70 (e.g., "Marriage" + "Long-term")
- Default: 30

Bonus Multipliers:

Profile Completeness Bonus:

- 90%+: 1.2x
- 75%+: 1.15x
- 60%+: 1.1x
- 40%+: 1.05x

Activity Bonus:

- Active in last hour: 1.1x

- Active in last 24h: 1.0x
- Active in last week: 0.95x
- Inactive >1 week: 0.8x

🔌 API Endpoints

Discovery Endpoints

1. Get Discovery Users

```
GET /api/discover/:userId?limit=20&offset=0&minScore=30&sortBy=score
```

Authentication: Required (Bearer token)

Query Parameters:

- `limit` (default: 20) - Number of users to return
- `offset` (default: 0) - Pagination offset
- `minScore` (default: 30) - Minimum compatibility score
- `sortBy` - Sort method: `score`, `distance`, `recent`, `active`

Response:

```
json
```

```
{
  "success": true,
  "users": [
    {
      "_id": "user_id",
      "username": "johndoe",
      "profilePicture": "url",
      "photos": ["url1", "url2"],
      "bio": "Bio text",
      "age": 25,
      "compatibilityScore": 87,
      "distance": 5,
      "interests": ["hiking", "photography"],
      "relationshipGoals": "Long-term relationship",
      "gender": "Male",
      "location": "New York, USA"
    }
  ],
  "total": 50,
  "hasMore": true
}
```


Algorithm:

1. Excludes users already interacted with
2. Applies gender preference filter
3. Applies age range filter
4. Calculates compatibility scores
5. Filters by minimum score
6. Applies distance filter
7. Sorts by preference
8. Paginates results

9. Returns only visible fields based on privacy settings

---

2. Handle Interaction

```
POST /api/discover/:userId/interact
```

Authentication: Required

Body:

```json
{
  "targetUserId": "user_id",
  "action": "like" // "like", "pass", "super_like", "block"
}
```

Response:

```json
{
  "success": true,
  "interaction": {
    "_id": "interaction_id",
    "fromUser": "user_id",
    "toUser": "target_user_id",
    "status": "liked",
    "matchScore": 87
  },
  "isMatch": false
}
```

If Mutual Match:

```json
{
  "success": true,
  "interaction": {...},
  "isMatch": true,
  "match": {
    "user": {
      "_id": "user_id",
      "username": "janedoe",
      "profilePicture": "url",
      "bio": "Bio",
      "age": 24
    },
    "compatibilityScore": 87
  }
}
```

Logic:

1. Validates action and users exist
2. Checks for duplicate interaction (409 error if exists)
3. Calculates compatibility score
4. Creates interaction record
5. Checks for reverse interaction (mutual match)
6. If match found, updates both records to "matched"
7. Returns match details if mutual

---

3. Get Matches

```
GET /api/discover/:userId/matches?limit=50
```

Authentication: Required

Response:

```json
{
  "success": true,
  "matches": [
    {
      "matchId": "match_id",
      "user": {
        "_id": "user_id",
        "username": "janedoe",
        "profilePicture": "url",
        "bio": "Bio",
        "age": 24,
        "location": "New York, USA"
      },
      "compatibilityScore": 87,
      "matchedAt": "2024-01-15T10:30:00.000Z",
      "conversationStarted": false,
      "lastMessageAt": null
    }
  ],
  "total": 15
}
```

---

4. Get Likes (Who Liked Me)

```
GET /api/discover/:userId/likes
```

Authentication: Required

Response:

```json
{
  "success": true,
  "likes": [
    {
      "likeId": "like_id",
      "user": {
        "_id": "user_id",
        "username": "johndoe",
        "profilePicture": "url",
        "bio": "Bio",
        "age": 26,
        "location": "Boston, USA",
        "interests": ["music", "travel"]
      },
      "isSuperLike": false,
      "likedAt": "2024-01-15T08:20:00.000Z"
    }
  ],
  "total": 8
}
```

---

5. Update Match Preferences

```
PUT /api/discover/:userId/preferences
```

Authentication: Required

Body:

```json
{
  "ageRange": {
    "min": 21,
    "max": 35
  },
  "distanceRange": 25,
  "genderPreference": ["Women", "Non-binary"],
  "dealBreakers": ["smoking"],
  "mustHaves": ["exercise"]
}
```

Response:

```json
{
  "success": true,
  "message": "Preferences updated successfully",
  "preferences": {...}
}
```

---

6. Update Discovery Settings

```
PUT /api/discover/:userId/settings
```

Authentication: Required

Body:

```json
{
  "isActive": true,
  "ageRangeVisible": true,
  "distanceVisible": false,
  "lastActiveVisible": true
}
```

---

7. Get Discovery Statistics

```
GET /api/discover/:userId/stats
```

Authentication: Required

Response:

```json
{
  "success": true,
  "stats": {
    "likesReceived": 45,
    "likesSent": 32,
    "totalMatches": 12,
    "passes": 120,
    "recentMatches": 3
  }
}
```

---

8. Unmatch

```
POST /api/discover/:userId/unmatch
```

Authentication: Required

Body:

```json
{
  "targetUserId": "user_id"
}
```

Response:

```json
{
  "success": true,
  "message": "Unmatched successfully"
}
```

📁 File Structure

```
backend/
├── config/
│   ├── db_connection.js
│   ├── mailer.js
│   └── cloudinary.js
│
├── controllers/
│   ├── authController.js
│   ├── profileController.js
│   └── discoverController.js    ⭐ NEW
```

```
│
├── middleware/
│   └── auth.js                ⭐ NEW
│
├── models/
│   ├── User.js                🔄 ENHANCED
│   └── Match.js               ⭐ NEW
│
├── routes/
│   ├── auth.js
│   ├── profile.js
│   └── discover.js            ⭐ NEW
│
├── utils/
│   └── matchingAlgorithm.js       ⭐ NEW
│
├── package.json
└── server.js                  🔄 UPDATED
```

🚀 Setup Instructions

1. Copy New Files

Copy these files to your backend directory:

- `models/User.js` (replace existing)
- `models/Match.js` (new)
- `middleware/auth.js` (new)
- `controllers/discoverController.js` (new)
- `routes/discover.js` (new)
- `utils/matchingAlgorithm.js` (new)

- `server.js` (replace existing)

2. Database Migration

The enhanced User model is backward compatible, but you should:

```javascript
```

```
// Run this script to update existing users
import User from './models/User.js';

async function migrateUsers() {
  const users = await User.find({});

  for (let user of users) {
    // Calculate age if dateOfBirth exists
    if (user.dateOfBirth && !user.age) {
      user.age = user.calculatedAge;
    }

    // Initialize new fields
    if (!user.matchPreferences) {
      user.matchPreferences = {
        ageRange: { min: 18, max: 100 },
        distanceRange: 50,
        genderPreference: [],
        dealBreakers: [],
        mustHaves: []
      };
    }

    if (!user.discoverySettings) {
      user.discoverySettings = {
        isActive: true,
        ageRangeVisible: true,
        distanceVisible: true,
        lastActiveVisible: true
      };
    }

    await user.save();
```

```
  }
}
```

3. Environment Variables

Ensure your `.env` has:

```
JWT_SECRET=your_secret_key
MONGODB_URI=your_mongodb_uri
NODE_ENV=development
```

4. Test the Setup

```bash
# Start server
npm run dev

# Test discovery endpoint
curl http://localhost:5000/api/discover/{userId} \
  -H "Authorization: Bearer YOUR_JWT_TOKEN"

# Test interaction
curl -X POST http://localhost:5000/api/discover/{userId}/interact \
  -H "Authorization: Bearer YOUR_JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"targetUserId": "target_id", "action": "like"}'
```

🔒 Security Features

1. **JWT Authentication** - All discovery routes protected
2. **Privacy Controls** - Field-level visibility settings

3. **Duplicate Prevention** - Compound indexes prevent duplicate interactions
4. **Input Validation** - Mongoose validators on all fields
5. **Error Handling** - Comprehensive error responses

🎯 Key Features

✅ Smart Matching Algorithm

- Multi-factor compatibility scoring
- Profile completeness bonuses
- Activity-based ranking

✅ Efficient Queries

- MongoDB indexes on critical fields
- Pagination support
- Distance-based geo queries

✅ Privacy First

- Granular visibility controls
- Only shows users who opted-in to discovery
- Respects user preferences

✅ Real-time Matching

- Instant mutual match detection
- Match notifications ready

✅ Scalable Design

- Handles thousands of users

- Optimized database queries
- Ready for caching layer

📊 Next Steps

Once frontend is ready, you can add:

1. **Push Notifications** - For new matches and likes
2. **Messaging System** - In-app chat
3. **Advanced Filters** - Education level, height, etc.
4. **Boost Feature** - Paid visibility increase
5. **Daily Limits** - Free tier restrictions
6. **Analytics** - Match success tracking

🐛 Debugging

No users in discovery?

- Check `discoverySettings.isActive` is true
- Verify age range preferences
- Check if all users have been interacted with

Low compatibility scores?

- Ensure users have filled profiles
- Check interest tags are set
- Verify location data exists

Matches not detecting?

- Check both users have liked each other
- Verify Match model indexes are created
- Check for duplicate prevention logic

📚 Resources

- MongoDB Geospatial Queries: https://docs.mongodb.com/manual/geospatial-queries/
- Haversine Formula: https://en.wikipedia.org/wiki/Haversine_formula
- Jaccard Similarity: https://en.wikipedia.org/wiki/Jaccard_index

---

Backend is ready! Move to frontend implementation next. 🚀